

SMABFGS - IK Solver Explanation:

① Optimization Variables

If we take Puma 560 as Example.

it have 6 joints

$$q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \end{bmatrix}$$

optimization variable.

→ why
Because we change this values for better solution. we are optimizing ~~using~~ by changing 'q'

② Forward Kinematics (FK)

we will be having Joint Angles using the Joint Angles we will be calculating End Effector position.

$q \rightarrow$ Forward Kinematics $\rightarrow T(q)$

$$T_0^6(q) = A_1(q_1)A_2(q_2)\dots A_6(q_6)$$

$$T_{\text{Target}} = \begin{bmatrix} R_{\text{Target}} & P_{\text{Target}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

How does Robot know it is Improving $\rightarrow$??

④ Objective Function

* Error:

ex

$$P_{\text{Target}} = \begin{bmatrix} 0.632 \\ 0.050 \\ 1.288 \end{bmatrix}$$

and Robot has a particular Joint configuration

$$q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \end{bmatrix}$$

After using this we get a position -

$$P(q) = \begin{bmatrix} 0.600 \\ 0.070 \\ 1.250 \end{bmatrix}$$

Position error

$$e_p(q) = P(q) - P(\text{Target})$$

$$e_p = \begin{bmatrix} -0.032 \\ 0.020 \\ -0.038 \end{bmatrix}$$

Orientation Error :-

Position error alone is not sufficient so

$$e_o(q) = \log(R_{\text{target}} R(q)^T)$$

$$e_o \in \mathbb{R}^3$$

The solver Needs one single Numerical score to evaluate each candidate Joint configuration.

The score is called Objective Function

$$f(q)$$

$$f(q) = \underbrace{w_p \frac{1}{2} \|e_p(q)\|^2}_{\text{Position contribution}} + \underbrace{w_o \frac{1}{2} \|e_o(q)\|^2}_{\text{orientation contribution}} + \underbrace{w_{\text{limit}} f_{\text{penalty}}(q)}_{\text{Penalizes configurations that approach or violate 'joint' limits, depending on implementation}}$$

Position contribution



Measures squared position error.

orientation contribution



Measures squared orientation error

Penalizes configurations that approach or violate 'joint' limits, depending on implementation

The objective Function uses Relative weights and combines them

$$w_p = 1.00$$

$$w_o = 0.20$$

SMA Population Initialization:-

$q \rightarrow$ Represents joint angles

$f(q) \rightarrow$ Represents how good a configuration is.

SMA algorithm creates a population of candidate solutions



Instead of Trying on ~~the~~ Posture at a time it generates multiple possible Postures.

Ex:- $N=10$

candidate	J_1	J_2	J_3 .. J_6
1	10	20	-15
2	30	40	5
⋮			
10	-20°	15°	25°

Mathematically

$X_i =$



one candidates solution.

$\begin{bmatrix} q_{i1} \\ q_{i2} \\ q_{i3} \\ q_{i4} \\ q_{i5} \\ q_{i6} \end{bmatrix} \rightarrow$ candidates index.
 $\in \mathbb{R}^6$

How are candidates generated

The solver uses Joint limits.

$X_i \sim$ Random values within Joint Bounds

For our Application Random seed = 42

↓
To make Random initialization
Reproducible.

What Happens After Initialization



① Once generation of candidates, The solver evaluates every candidate using the Objective Function. $f(X_i)$

The Algorithm Identifies the candidate with smallest Objective values as the current Best

$$X_B = \arg \min_i f(X_i)$$

↓
current Best candidate.

SMA Global Exploration

How does SMA move the candidates to find Better Joint Angles.

① using objective function $\rightarrow$ finding best candidate.

② Exploration vs Exploitation

Exploration

search new, unexplored solution

Exploitation :-

search around a promising Region.

why does SMA Need Both

* if every candidate moves toward Promising region, the population might lose diversity and focus on one Region too early.

* If every candidate moves randomly, it may struggle to refine good solutions

How SMA update a candidate

* SMA calculates certain Adaptive parameters and Random values

Then it selects an update Rule.

A. Exploration Jump.

generate new random configuration within Joint Bounds

B. Food Attraction Movement

Move using the current best candidate, other population candidates and calculation weights.

C. Shrinkage Movement

scale the candidate's current coordinates using a Random search vector.

SMA updates candidates to search for lower objective values.
~~SMA~~

Exploration Jump

~~instead of~~ Helps algorithm search completely new regions of the robot's joint space.

when does it happen

The Algorithm generates Random Number

$$r_i \in [0, 1]$$

then it checks

$$r_i < \bar{r}$$

In my implementation $\rightarrow \bar{r} = 0.03$

if $r_i < \bar{r} \rightarrow$ exploration Triggered

else $\rightarrow$ Not Triggered.

Approximately 3% chance that a candidate will perform exploration Jump.

What Happens During the Jump:-

$$X_i(t+1) = \underset{\substack{\downarrow \\ \text{lower limit} \\ \text{of Joint}}}{LB} + \underset{\substack{\text{Random value} \\ \text{from 0 to 1}}}{\text{rand}} (UB - LB)$$

Ex-

$$q_1 \cdot -180^\circ \leq q_1 \leq 180^\circ$$

$$\text{Random, vale} = 0.75$$

$$\begin{aligned} q_1 &= -180 + 0.75 (180 + 180) \\ &= -180 + 270 \\ &= 90 \end{aligned}$$

$\boxed{q_1 = 90^\circ}$ $\rightarrow$ In same way for all the Joints.

SMA Adaptive Parameters

1st Parameter : Shrinkage Control

$$b = 1 - \frac{t}{T_{\max}}$$

→ current iteration

→ max iterations.

Your configuration

$$T_{\max} = 40$$

lets calculate it

Iteration	b
0	1.00
10	0.75
20	0.50
30	0.25
40	0.00

As iterations progress
b decreases from 1 to 0

why shrinkage movement uses

$$V_c \sim U(-b, b)$$

This means Random value V_c is generated
between $-b$ and $+b$

B controls the range of random shrinkage factor.

2nd Parameter

Search control

$$a = \tanh^{-1} \left(1 - \frac{t}{T_{\max}} \right)$$

a changes with the iteration number and is used to generate the search vector.

V_b

* a controls the scale of search vector V_b

SMA weight w_i

candidate	Objective value	Meang.
A	0.10	very good
B	0.50	Average
C	2.00	Poor

The weight w_i tells the algorithm how strongly a candidate robot configuration should influence the next search.

smaller value $\rightarrow$ Better.

A candidate with low objective value will receive a weight that encourages the algorithm to learn from it.

How SMA generates a New Joint Configuration:-

It uses best solution X_B and another Randomly selected solution X_A to guide movement.

~~$X_i(t+1) = X_B + v_b (w_i X_A - X_i)$~~

$$X_i(t+1) = X_B + v_b (w_i X_A - X_i)$$

current candidates
Joint Angles

X_B = Best candidates Joint Angles.

v_b → movement control factor

w_i → weight of current candidate

X_A ⇒ Randomly selected candidate

X_B : Best solution

X_B = Configuration with the lowest objective value

$$X_B = [q_1, q_2, q_3, q_4, q_5, q_6]$$

X_A : Randomly selected solution

X_A → another candidate selected from population.

* It introduces variation and Diversity, preventing every candidate from moving in exactly the same way.

X_B → Guides candidates toward the best known solution

X_A → Adds variation

Food Attraction Movement

In SMA the food represents the best solution found so far.

Food = Robot Joint configuration with lowest Objective value.

$X_B \rightarrow$ Best Robot configuration.

A candidate compares its current position with best solution.

Movement direction $\propto X_B - X_i$
 $\hookrightarrow$ current position

Ex:

$$q_i = 40^\circ$$

$$q_B = 30^\circ \rightarrow \text{Best solution}$$

Now The candidate may move toward the

$$q_B \rightarrow 30^\circ$$

The two Adaptive Parameters a and b

$a \rightarrow$ controls the strength/range of one type of SMA movement

$b \rightarrow$ controls the striking/random movement as iterations progress.

Shrinkage Movement:

Imagine

we have candidate

X_i

and algorithm wants to adjust it.

* Instead of making completely random solution, SMA can shrink/adjust the candidate Around its current Region.

$$V_c \sim U(-b, b)$$

* It means generate a Random No. between $-b, +b$

Ex:

if $b = 0.5$

then V_c could be

$-0.3, 0.1, 0.45, -0.05$

etc..

As the iterations progress, b becomes smaller.

$$b = 1 - \frac{t}{T}$$

Complete SMA Iteration

One SMA iteration basically does this

Step-1

Take current ~~Pop~~ population of Robot configurations

Step-2

calculate objective value of Each one.

Step-3:-

Find Best configurations:

X_B

Step-4

For each candidate, decide how it should move

it can use

- ↳ Exploration
- ↳ ~~Exploitation~~ Food Attraction Movement.
- ↳ Shrinkage movement

Step-5

Generate new Joint configuration

Step-6

check its objective value again

Step-7

Keep Track of best solution

Then Repeat

BFGLS

* SMA Finds the best position right why do we need the BFGLS

Ex:-

After SMA finishes it finds

x_{SMA}
with very small objective value.

Ex

$$x_{SMA} = [205, 31.2, -15.7, 40.1, 50.8, 10.3]$$

This is a good solution.

But there might be a slightly better configuration nearby.

BFGLS $\rightarrow$ Broden - Fletcher - Goldfarb - Shanno.

$\hookrightarrow$ it is a local optimization Algorithm.

BFGLS $\rightarrow$ Refine the solution in that Area.

* It makes more precise adjustments to reach local minimum of the objective.

Gradient:-

it tells us which direction should we move to make objective decrease.

For

$$q = [q_1, q_2, q_3, q_4, q_5, q_6]$$

and

$f(q)$ $\rightarrow$ objective function

$$\nabla f(q) = \begin{bmatrix} \frac{\partial f}{\partial q_1} \\ \frac{\partial f}{\partial q_2} \\ \frac{\partial f}{\partial q_3} \\ \frac{\partial f}{\partial q_4} \\ \frac{\partial f}{\partial q_5} \\ \frac{\partial f}{\partial q_6} \end{bmatrix}$$

Understanding Gradient :-

Suppose we have an objective function

$$f(x, y) = x^2 + y^2$$

we want to minimize f

The minimum is obviously at

$$(x, y) = (0, 0)$$

because

$$0^2 + 0^2 = 0$$

Suppose we are currently at

$$(x, y) = 3, 4$$

$$f(3, 4) = 3^2 + 4^2 = 25$$

so, we are currently 25 units away from minimum in terms of this objective.

what does gradient tell us about:



we calculate partial derivatives

$$\frac{\partial f}{\partial x} = 2x$$

$$\frac{\partial f}{\partial y} = 2y$$

Therefore:

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

At (3, 4):

$$\nabla f(3, 4) = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

So the gradient is

$$\boxed{\nabla f = \begin{bmatrix} 6 \\ 8 \end{bmatrix}}$$

~~For~~ (6, 8) are the gradient values for coordinates (3, 4)

* Imagine standing on a hill middle.

The gradient tells us

* "If you move around from where you are standing, which direction makes the hill go upwards faster.

* It is like
I am currently here, which way should I move to make my error smallest.

What do we do with the gradient

~~$\alpha = 0.1$~~

suppose we take small step of
 $\alpha = 0.1$

Gradient $\rightarrow [6, 8]$

we want to reduce objective
so

$$[-6, -8]$$

Now multiply with timestep

$$[-6, -8](0.1) = [-0.6, -0.8]$$

our original point was

$[3, 4]$ so we add that

$$[3, 4] + [-0.6, -0.8]$$

$(2.4, 3.2) \rightarrow$ we went from $(3, 4)$ to $(2.4, 3.2)$

let us check

$$(2.4)^2 + (3.2)^2 = 5.76 + 10.24 \\ = 16$$

we went from 25 to 16

* we successfully Reduced Objective.

Here we have taken the
time step = 0.1 for ex

but BFGS does not do that.

Instead it learns from previous movements.
it observes:

I moved this much, and my gradient changed
this much.

From that information, BFGS builds an approximation
of the curvature.

Then on next iteration it can say:

"Based on what I learned about this objective
I should move in this direction, with this scaling."

* Basically the change in gradients gives BFGS
information about the shape of objective around
us.

BFGS Remembers

* where did I move

* How did the gradient change after I moved.

S = change in position

y = change in Gradient

$$S = (2.4, 3.2) - (3, 4) = (-0.6, -0.8)$$

$$y = (4.8, 6.4) - (6, 8) = (-1.2, -1.6)$$

what does the BFGS do with S, y

$$S = [-0.6, -0.8]$$

$$y = [-1.2, -1.6]$$

BFGS uses this information to learn how the

objective Behaves.

↳ we call that as Matrix $\rightarrow B$

why a Matrix

Our Robot have

$q_1, q_2, q_3, q_4, q_5, q_6$

So the curvature information is represented at

6×6 Matrix

we call this

How does B actually help choose the Next Direction?

What already know

∇f = current slope

$B \rightarrow$ BFGS learned information about the curvature

The search Direction is

$$p = -B \nabla f$$

Then we'll move

$$q_{\text{new}} = q + \alpha p$$

α determines how far we move.

we can write this as

$$q_{k+1} = q_k + \alpha P_k$$

New, New position

which direction

How many steps/

How far

Now we have completed one movement.

$$q_k \rightarrow q_{k+1}$$

we also calculate new Gradient.

So we Have

Old Information :-

$$q_k, \nabla f(q_k)$$

New Information :-

$$q_{k+1}, \nabla f(q_{k+1})$$

Now BFGS asks

"What did I learn from this movement."

① How much did we move

$$S_k = q_{k+1} - q_k$$

let

$$q_k = (3, 4)$$

$$q_{k+1} = (2.4, 3.2)$$

$$S_k = (-0.6, -0.8)$$

② Next, How much did gradient change

$$Y_k = \nabla f(q_{k+1}) - \nabla f(q_k)$$

Now BFGS learns

s_k
and

y_k

and uses them to update its matrix B .

BFGS update Formula

BFGS updates its matrix B using

$$\underbrace{B_{k+1}}_{\text{New } B} = \underbrace{B_k}_{\text{old } B} - \underbrace{\frac{B_k y_k y_k^T B_k}{y_k^T B_k y_k}}_{\text{old information}} + \underbrace{\frac{s_k s_k^T}{y_k^T s_k}}_{\text{New Information}}$$

New B = old B - old Information + new Information.